



GPU & CUDA Fundamentals

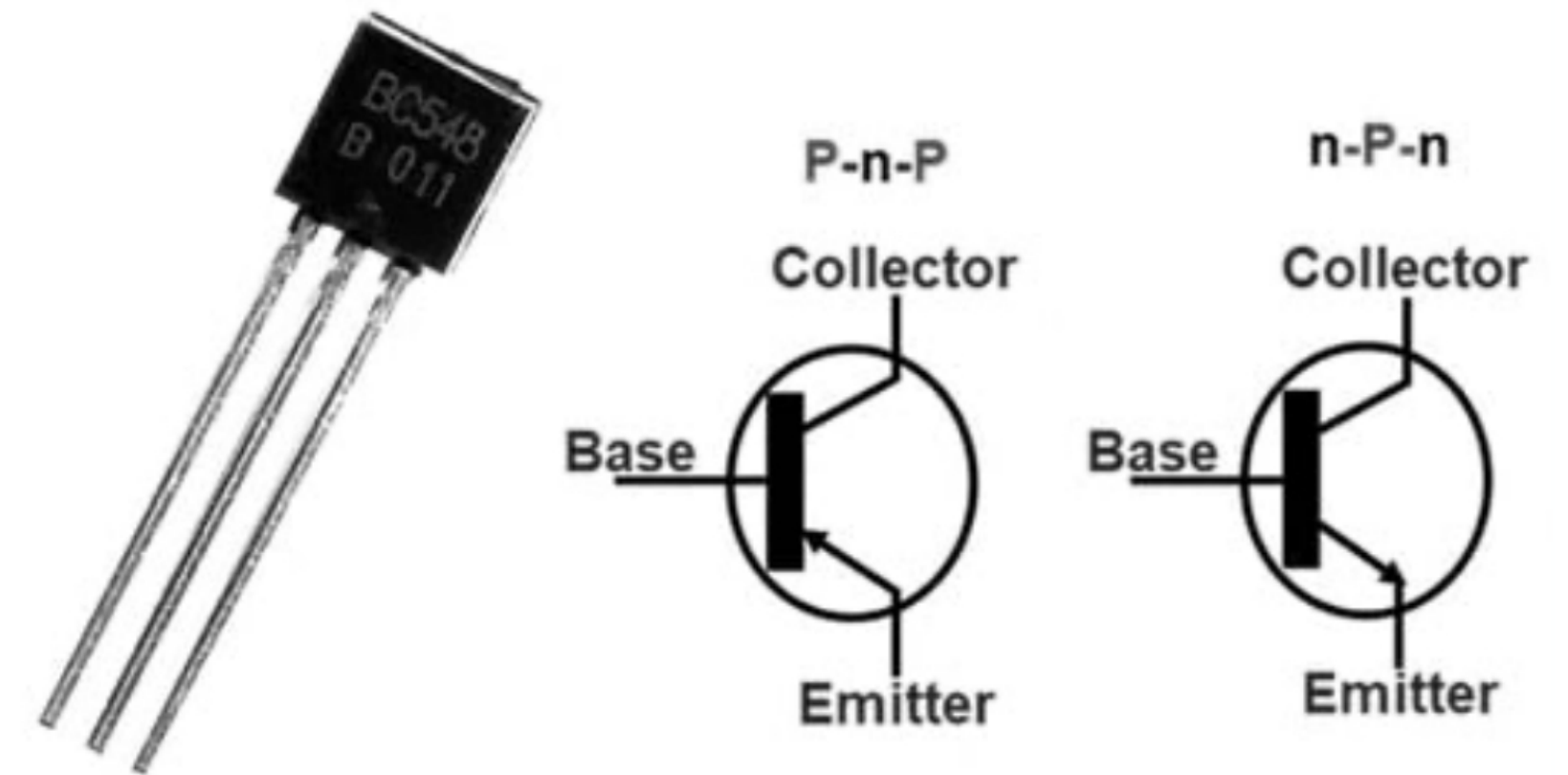
Введение в эффективные системы глубокого обучения

Великонивцев Федор Сергеевич, Yandex Research
Лекция 01, осень 2026

Transistor

- Electronic switch - controls electric current on a device
- Fundamental building block of all modern chips
- **Process size** - dimension of the smallest feature (e.g. *transistor*) $\Rightarrow$ can place more transistors on the same area

More transistors $\Rightarrow$ faster hardware



Moore's law

- Number of transistors on a chip doubles every 18 months
- As time goes by, process size decreases
- Number of transistor grows exponentially

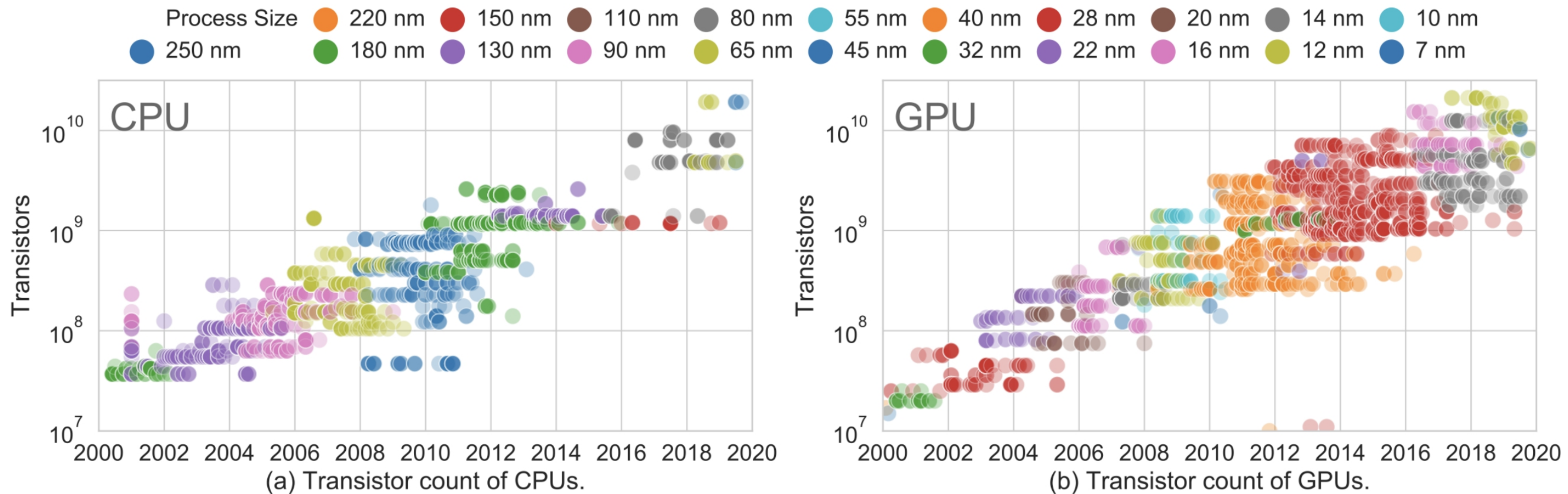


Fig. 1. Moore's Law is still valid for both CPUs and GPUs.

Dennard Scaling law

- Energy consumption per transistor reduces with decreasing process size **(a)**
- Power usage stays *kinda* constant in various process sizes **(b)**
- Clock frequency increases (hardware is faster) when you have more transistors per area block **(c)**

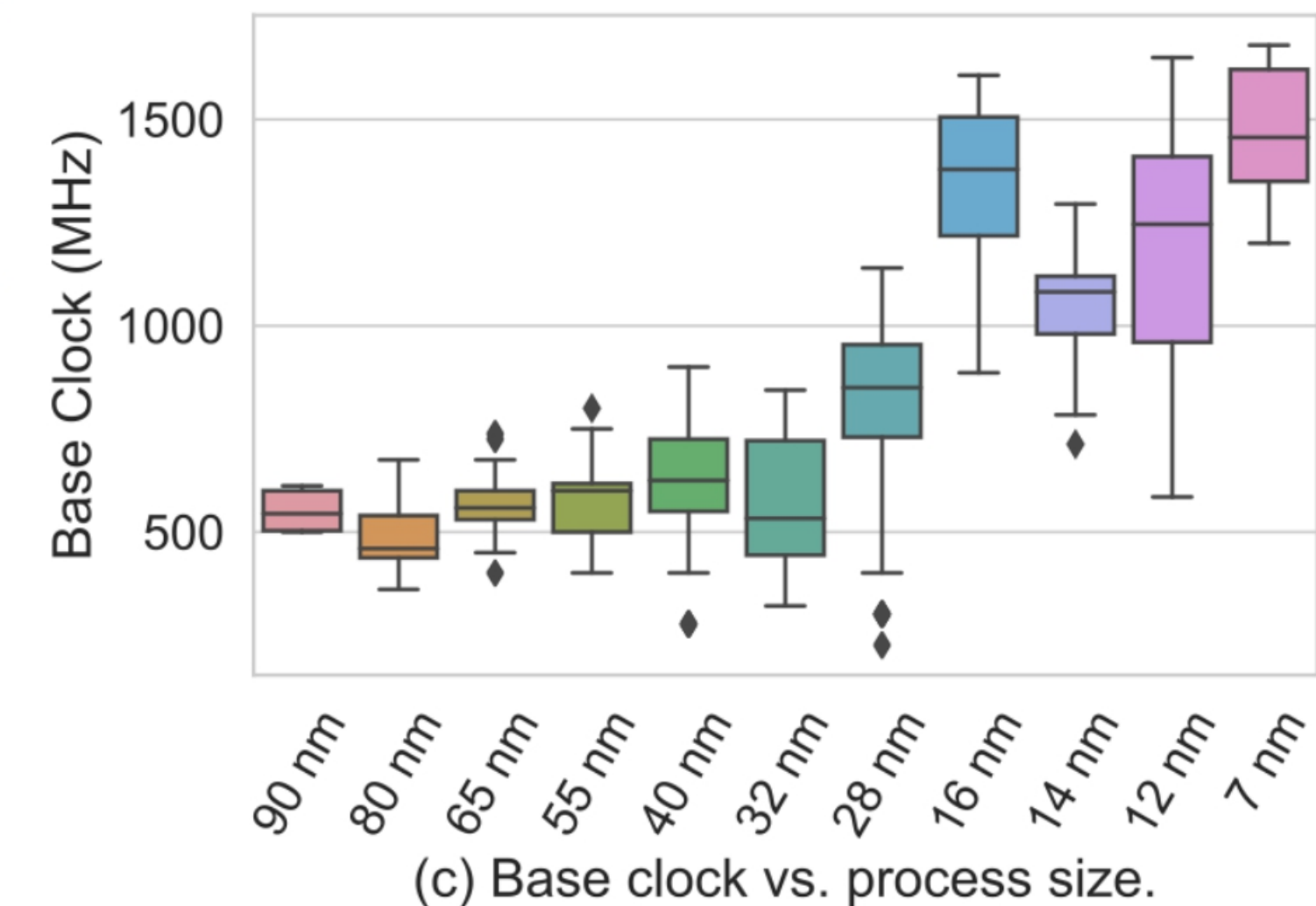
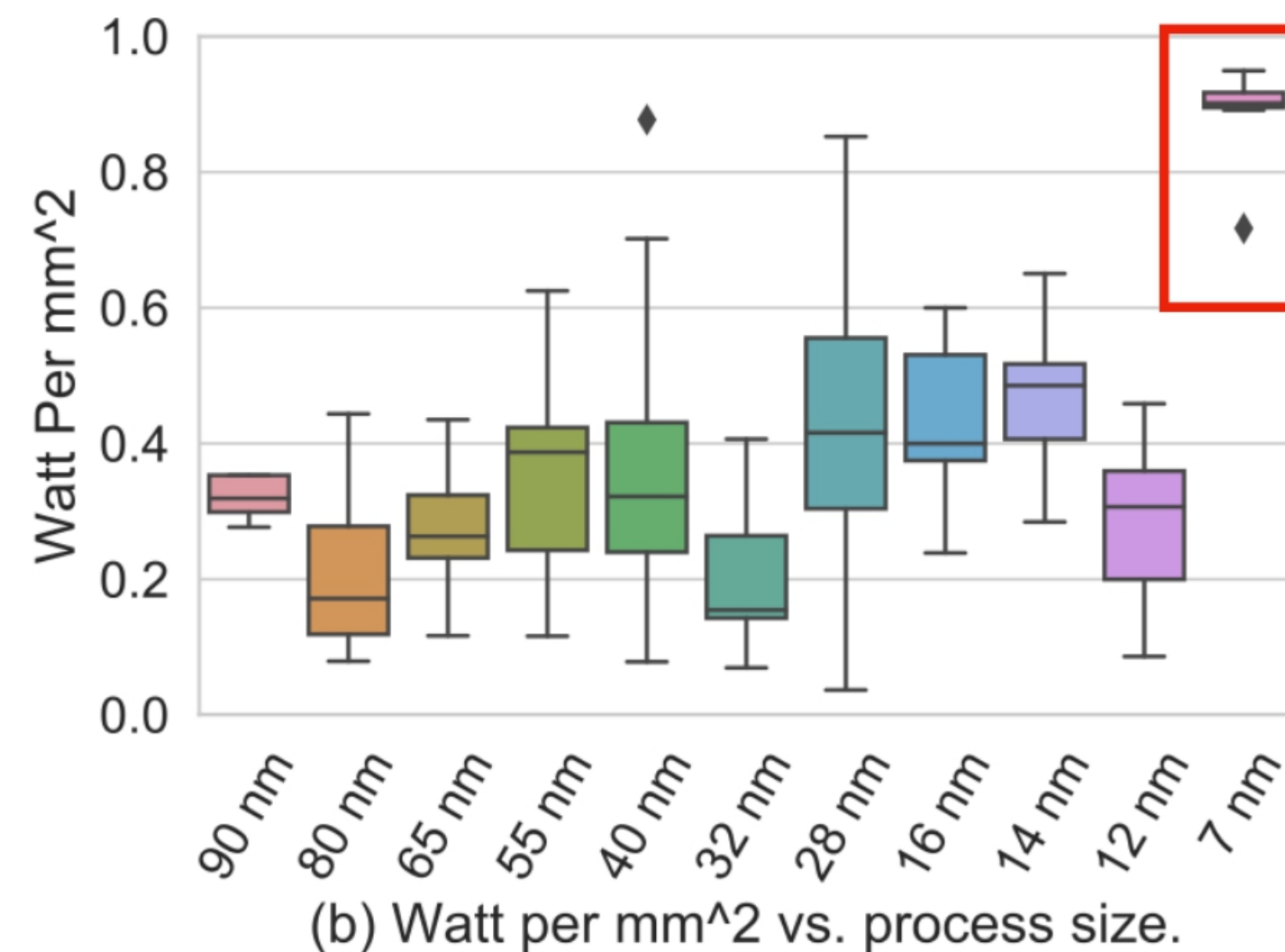
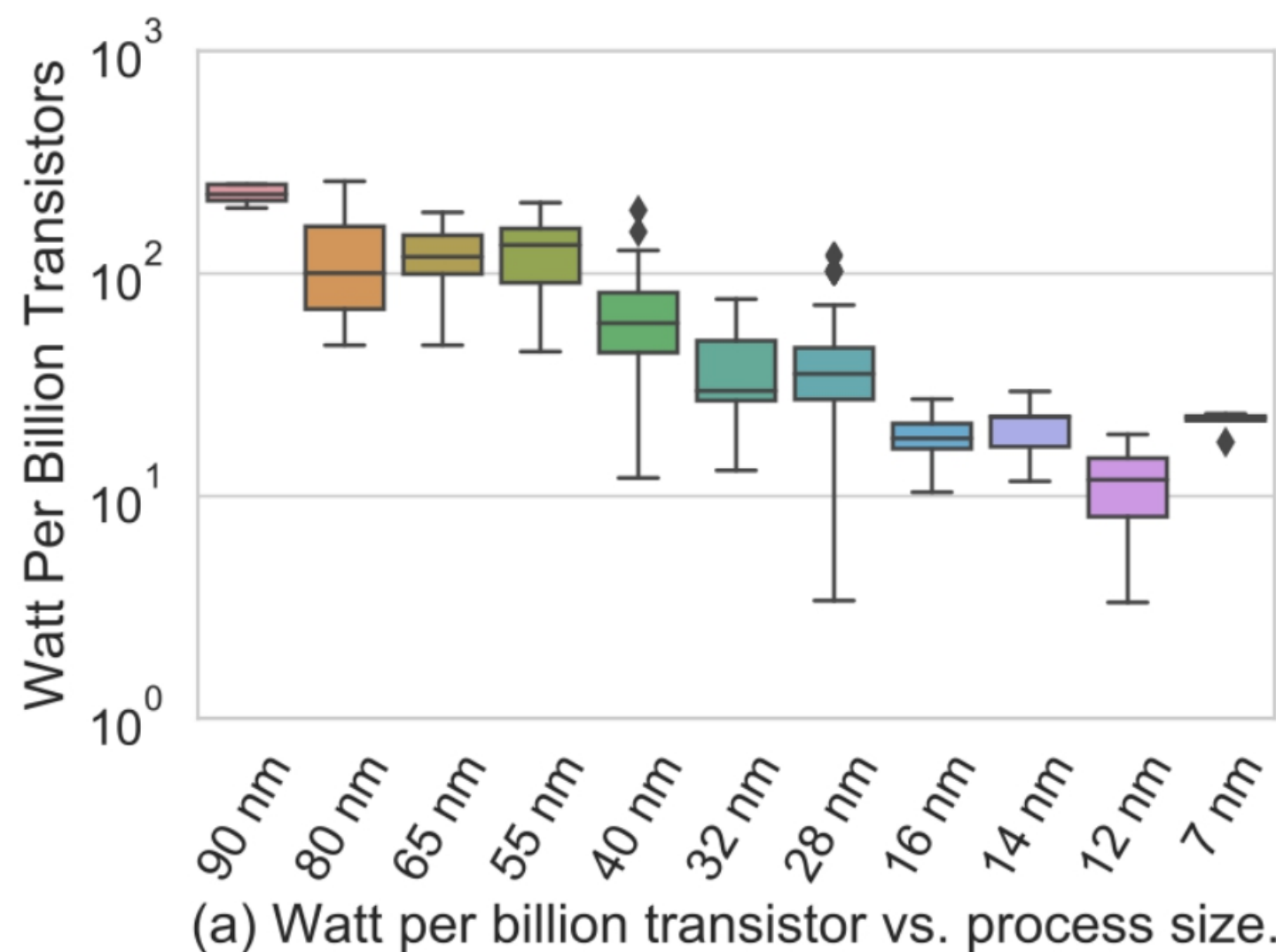


Fig. 4. Process Size vs. (a) Energy consumption per billion transistor. (b) Energy consumption per area. (c) Base clock speed.

* Figure represents characteristics of GPUs, but this is applicable for CPUs too

<https://arxiv.org/abs/1911.11313>

Fundamental Physical Limits to Transistor Size

- At 4-5nm scale transistors become affected by random electron fluctuations



Fundamental Physical Limits to Transistor Size

- At 4-5nm scale transistors become affected by random electron fluctuations
- Chip's circuits are manufactured with photolithography — you can't create circuits smaller half the frequency of the light wave (~10 nm)



Fundamental Physical Limits to Transistor Size

- At 4-5nm scale transistors become affected by random electron fluctuations
- Chip's circuits are manufactured with photolithography — you can't create circuits smaller half the frequency of the light wave (~10 nm)
- **Takeaway:** we are limited in scaling hardware performance



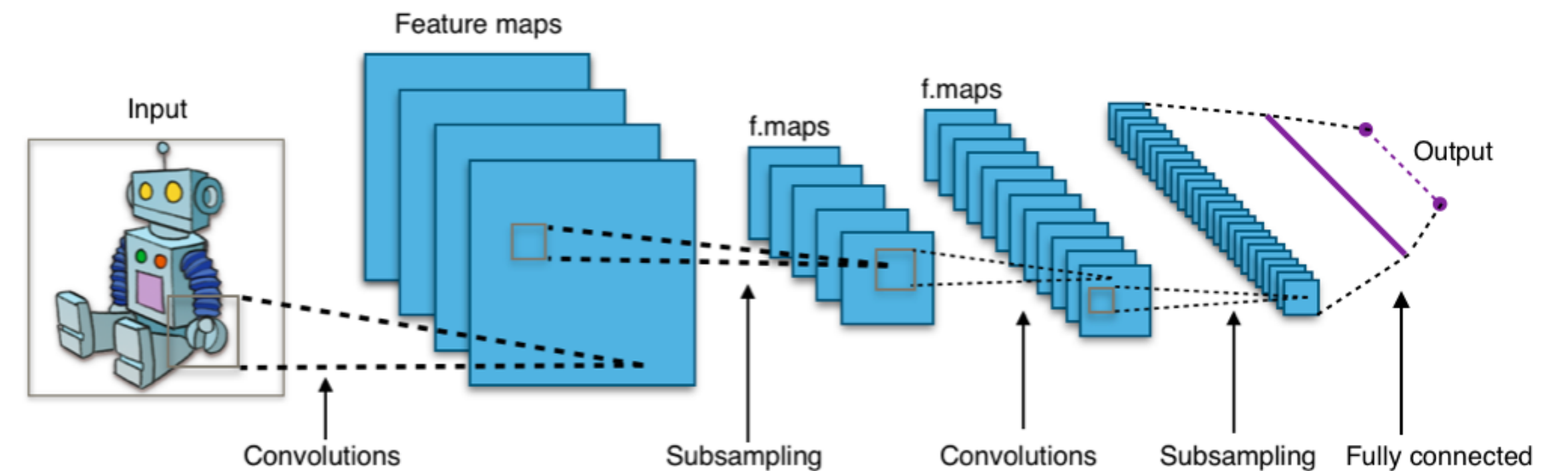
Fundamental Physical Limits to Transistor Size

- At 4-5nm scale transistors become affected by random electron fluctuations
- Chip's circuits are manufactured with photolithography — you can't create circuits smaller half the frequency of the light wave (~10 nm)
- **Takeaway:** we are limited in scaling hardware performance, **but we can optimize the parts we need for parallel computation**



Why GPUs for Deep Learning?

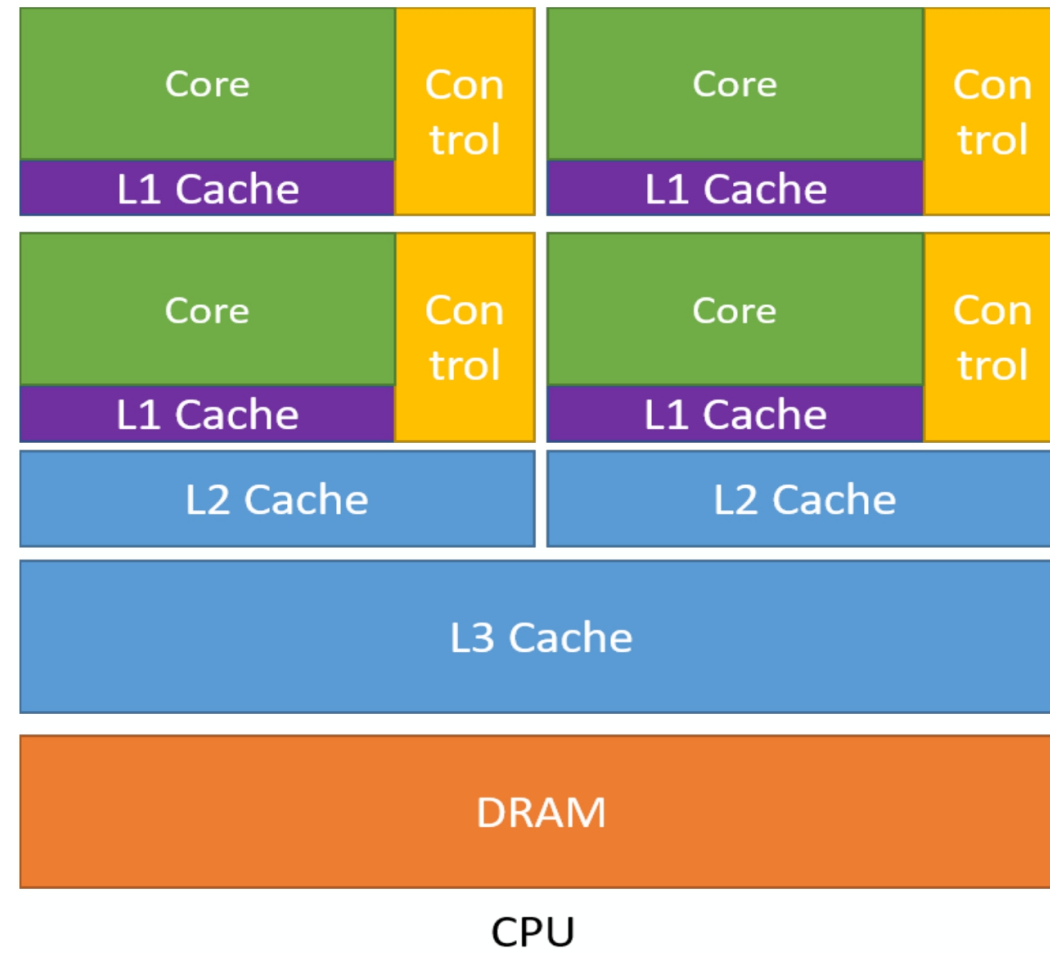
- DL exposes huge amounts of **similar, independent** numerical work:
 - Matrix multiplication: $A = CB$
 - Convolution: $K * A$
 - Attention: $\text{Softmax}(\mathbf{QK}^T)\mathbf{V}$
 - Vector operations\residuals, you name it



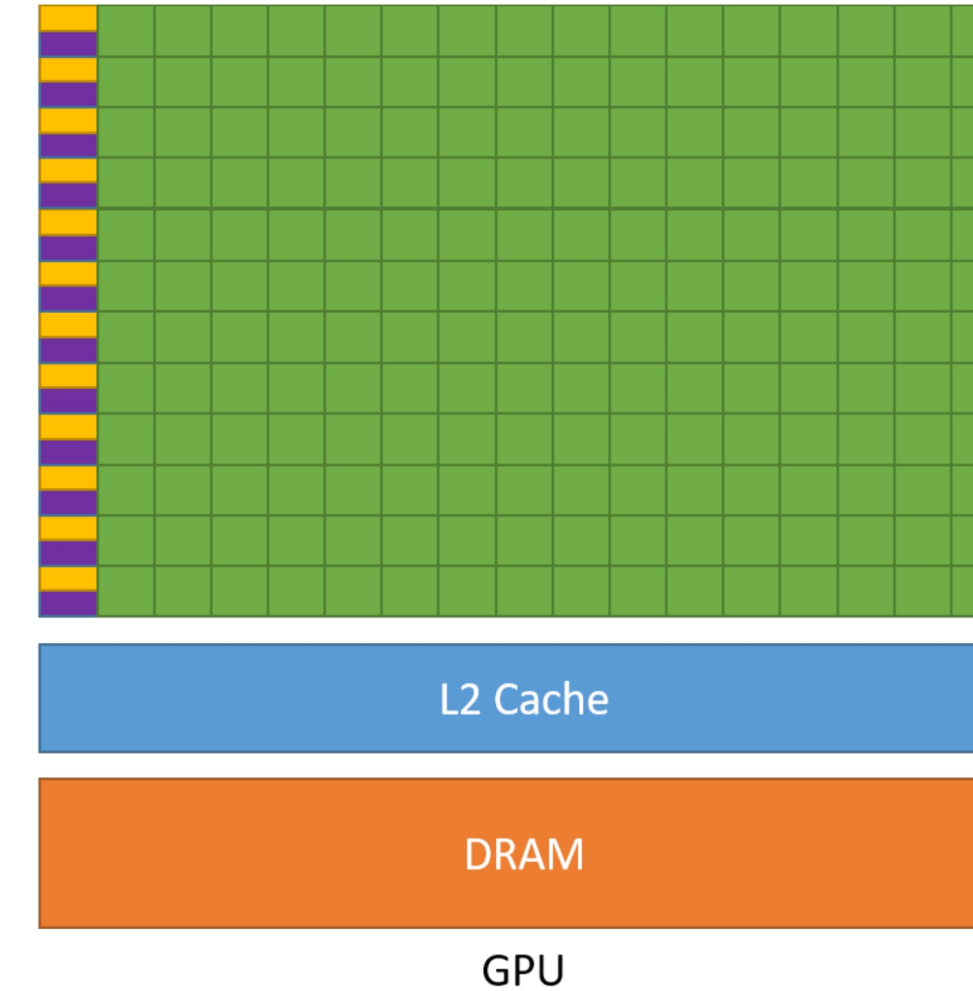
Why GPUs for Deep Learning?

- DL exposes huge amounts of **similar, independent** numerical work
- GPUs fit perfectly for these tasks!

CPUs vs GPUs: different design goals



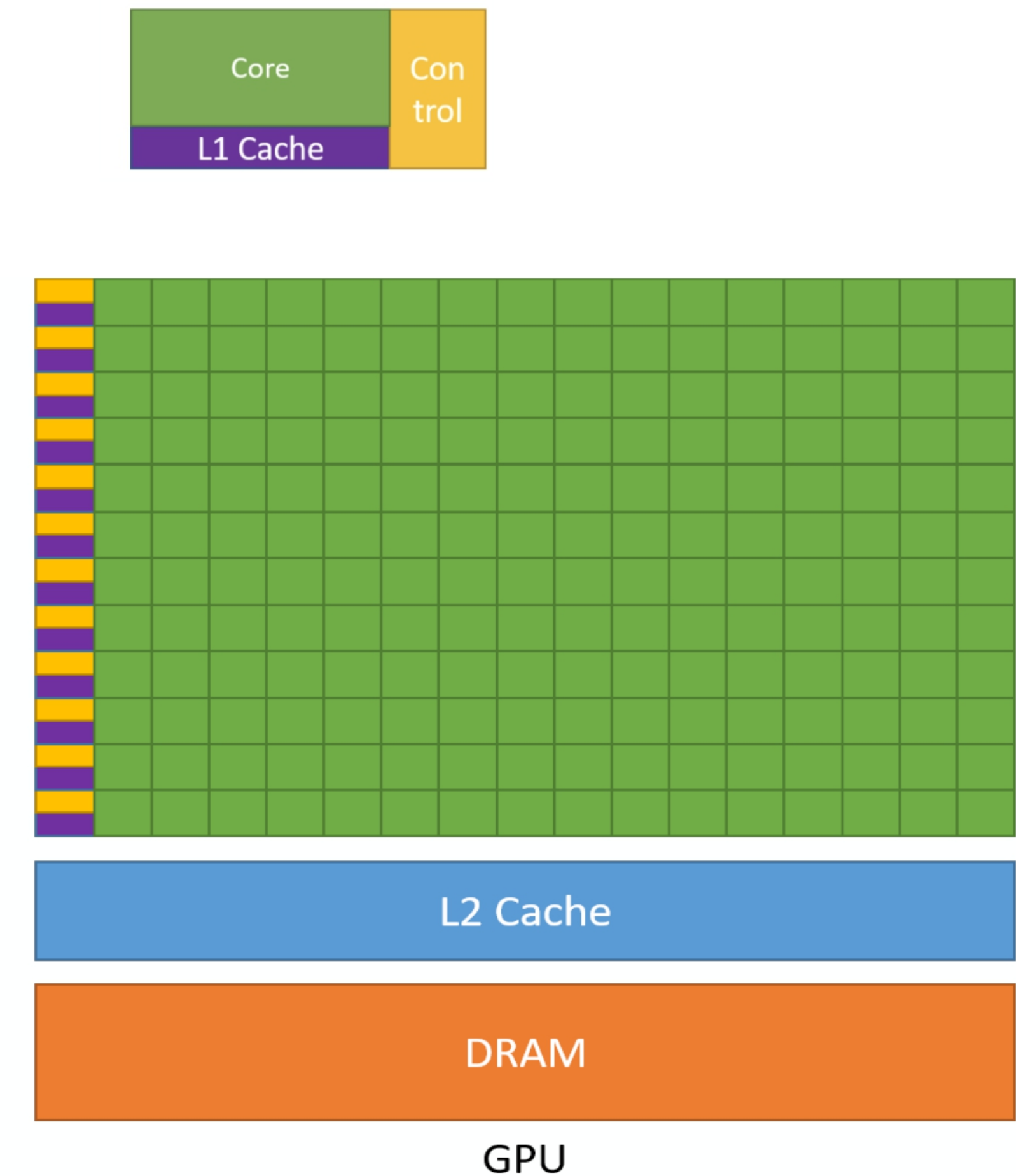
- Few powerful cores
- Large caches
- Sophisticated control (branch prediction/OoO execution)
- Excel at **irregular** or sequential work



- Many execution units, massive number of threads
- High Memory bandwidth, more transistors for data processing
- Shared control of [groups of] threads
- Excel at **simpler, regular** numerical work

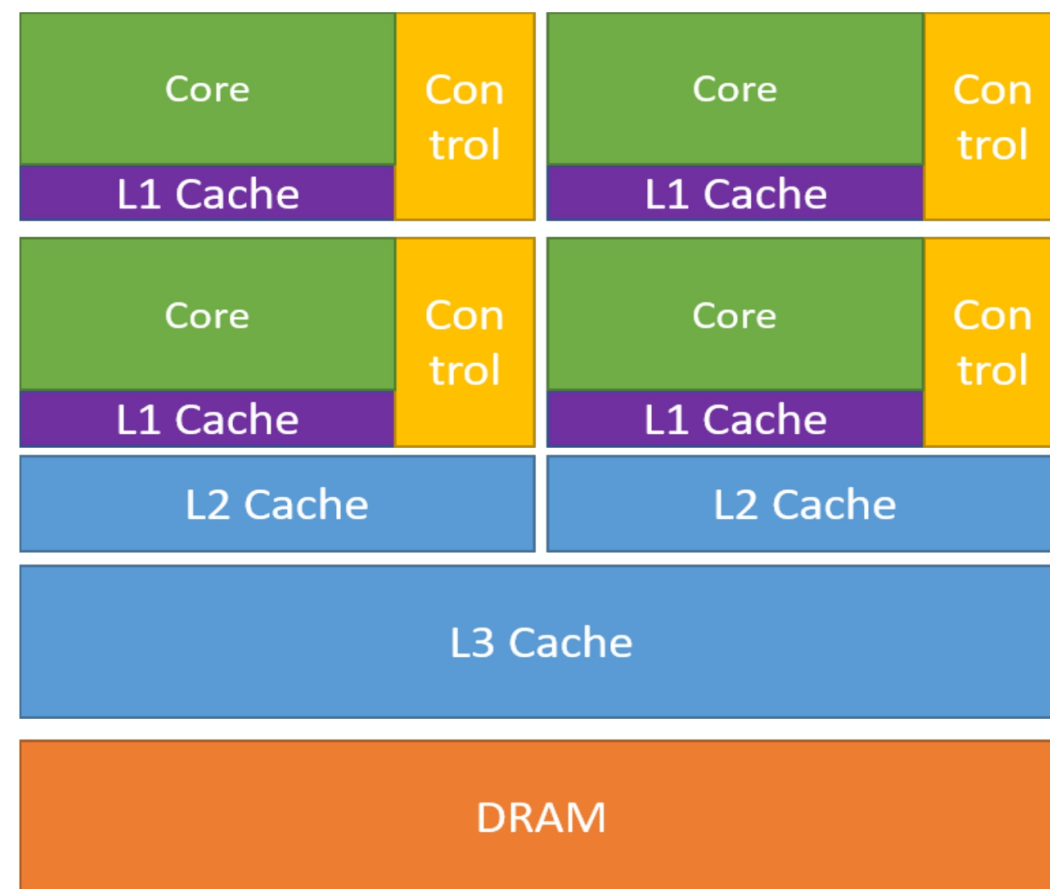
CPUs vs GPUs: different design goals

- **Massive parallelism**
 - Many execution units, massive number of threads
 - Large amount of independent numerical work on thousands of CUDA cores
 - The control of each thread is much simpler → more energy-efficient
- **High Throughput**
 - Much portion of the chip is dedicated to arithmetic throughput and high-bandwidth data movement
- **Hardware specialization**
 - Modern GPUs add matrix engines and data movement hardware to specifically target DL workloads



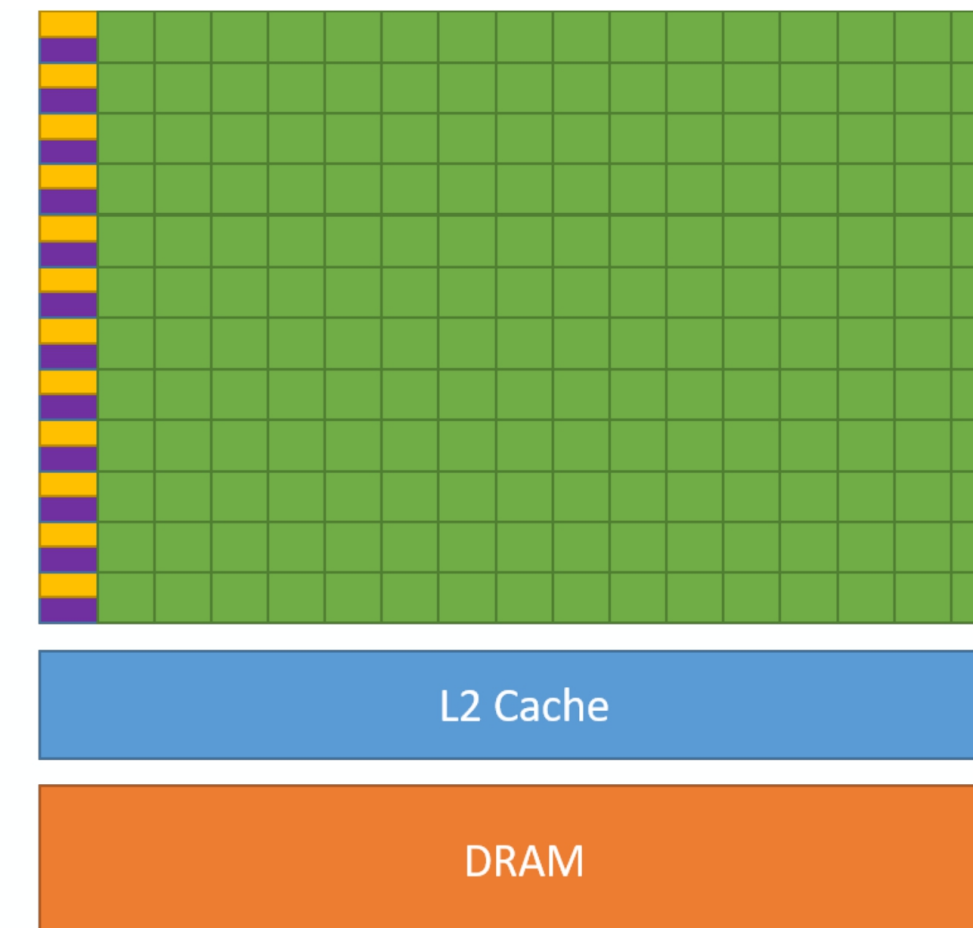
CPU vs GPU: different design goals

How much independent work can I expose?



CPU

Not so much **X**



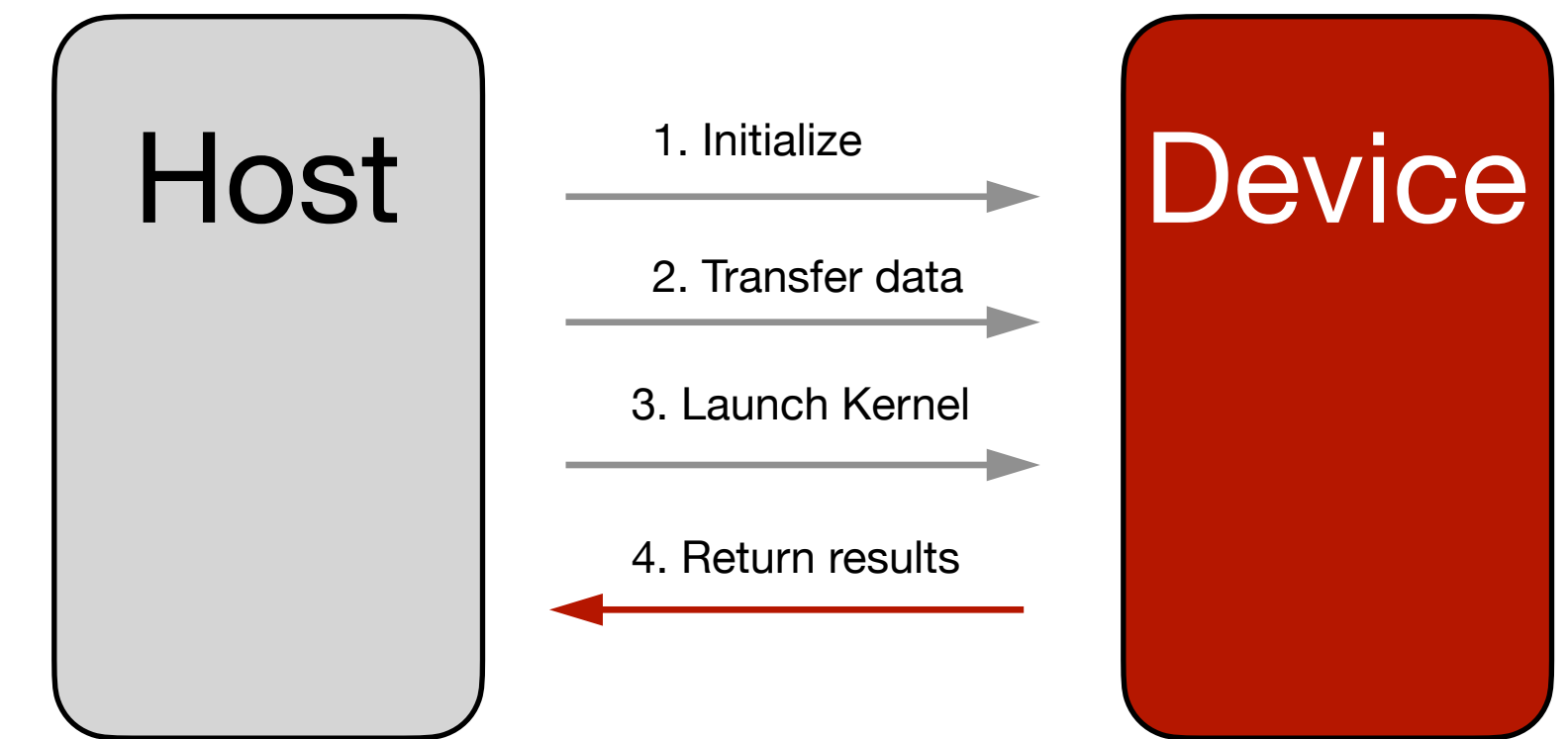
GPU

A lot! **✓**

GPU performance = parallelism + locality + specialization

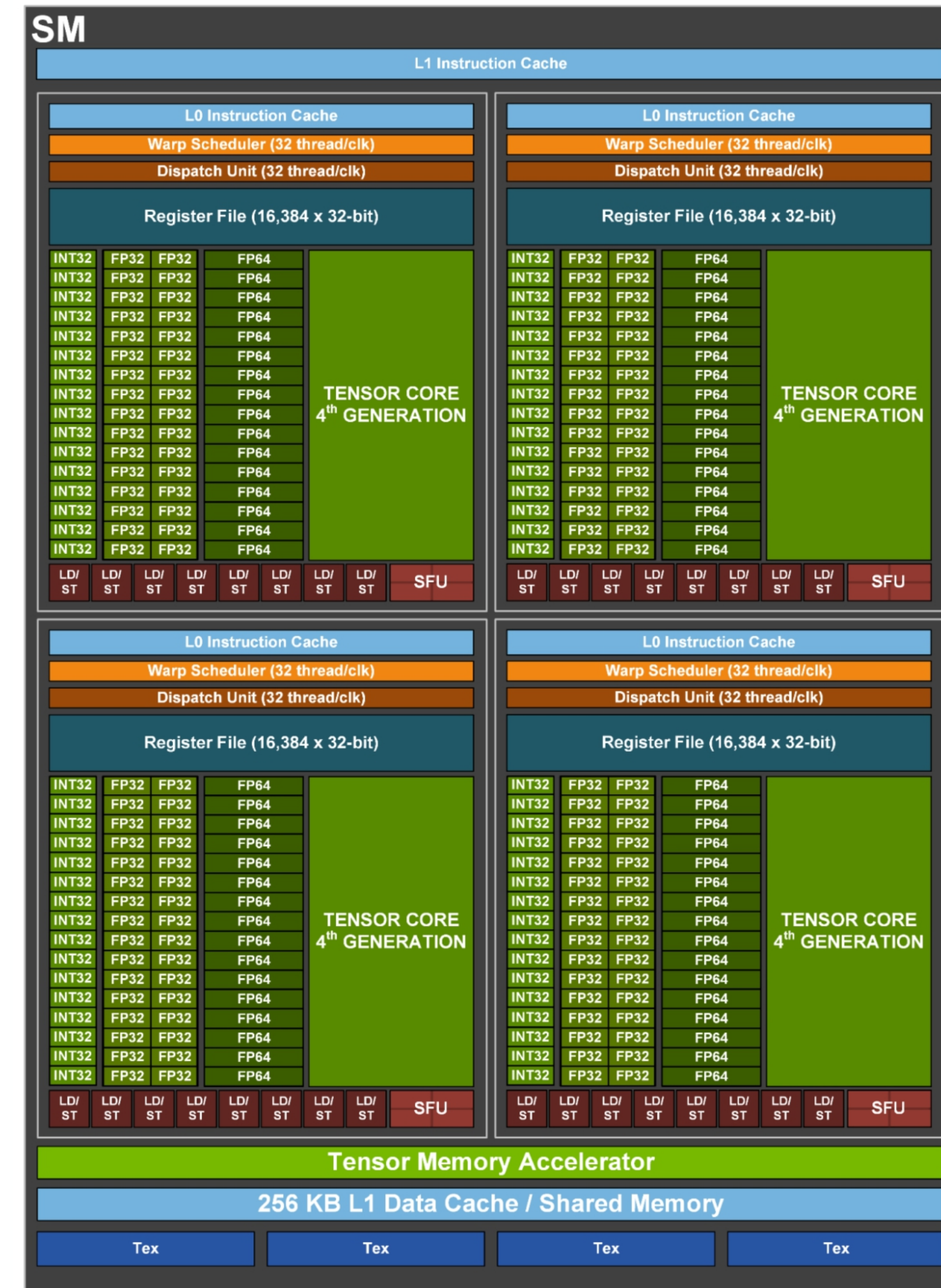
Host vs Device: who controls what?

- GPU (*Device*) is connected to CPU (*Host*)
- Host:
 - runs the application and framework
 - Allocates tensors and buffers
 - Launches *Device* work
- Device:
 - Executes kernels with massive parallelism on device-resident data



Streaming Multiprocessor (SM)

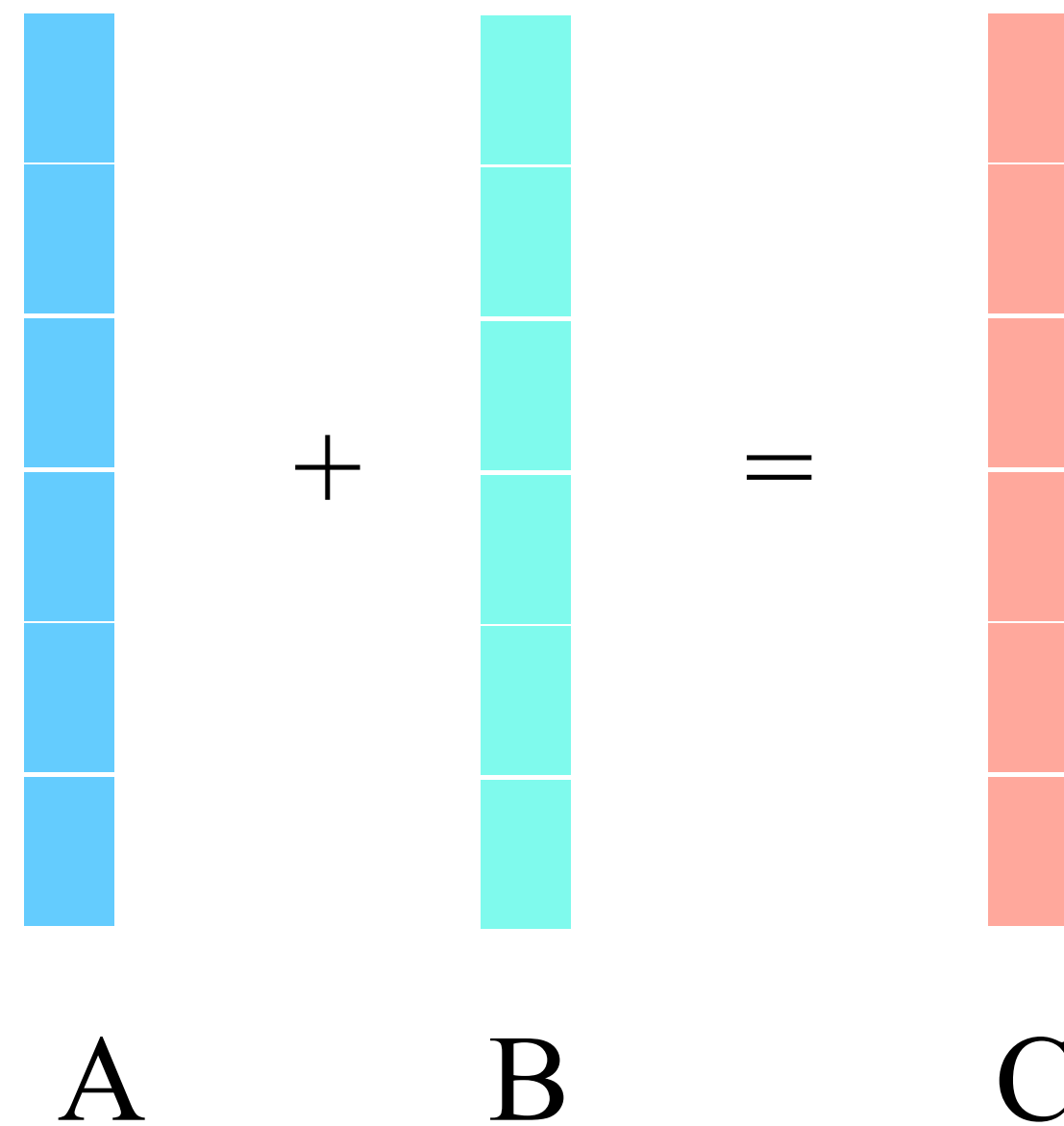
- Basic building unit of NVIDIA GPU
- Contains computational and data loading logic:
 - Arithmetical units
 - Registers
 - Warp schedulers (for issuing instructions)
 - Cache Units
 - Cooperation logic (shared memory)
 - Specialized units (Tensor Cores, Tensor Memory Accelerator)



CUDA Programming Model

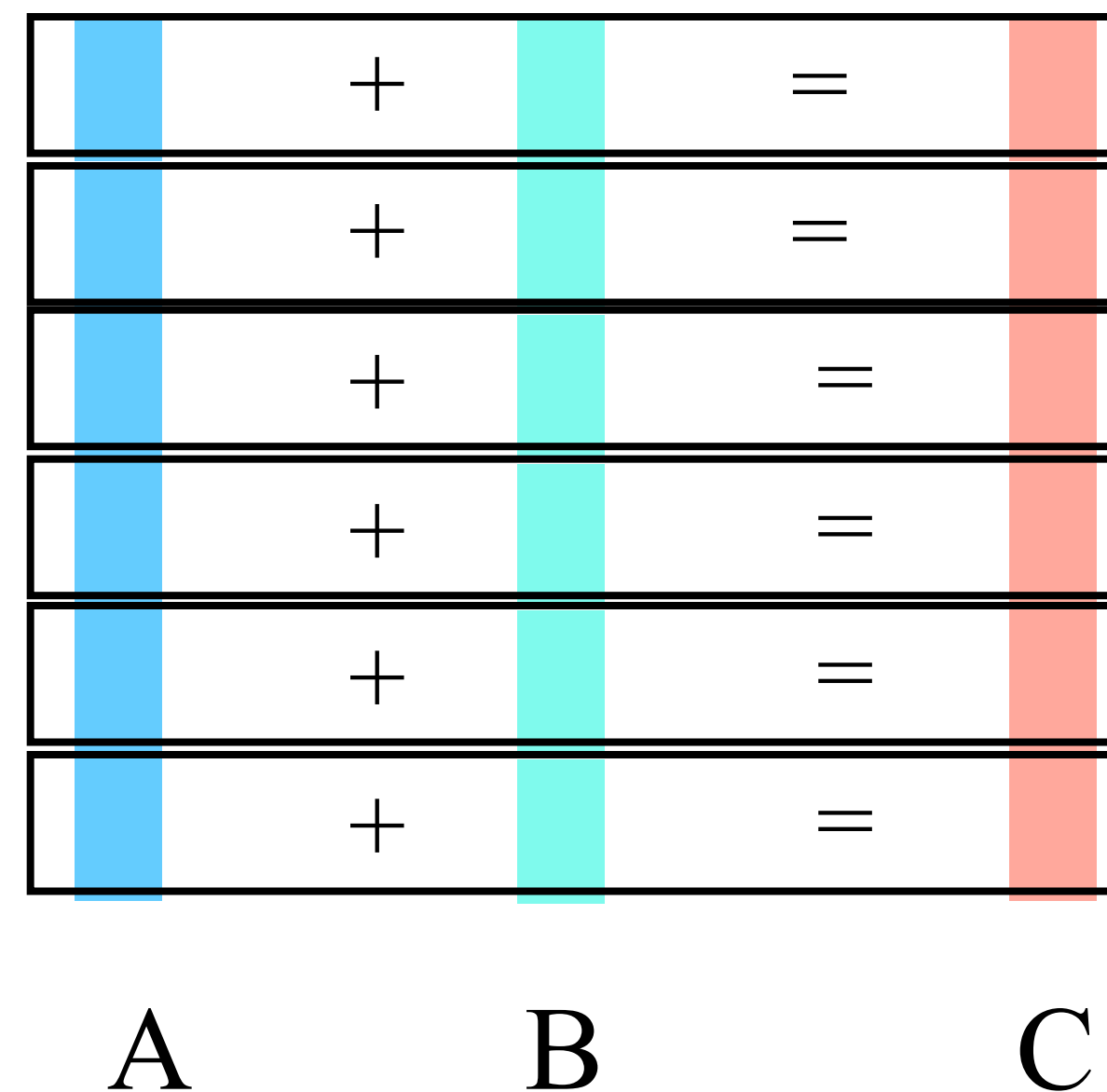
Kernel

- **Kernel** - a function executed on the GPU by many threads
- Each part computes a portion of the result:



Kernel

- **Kernel** - a function executed on the GPU by many threads
- Each part computes a portion of the result (same program, different data):

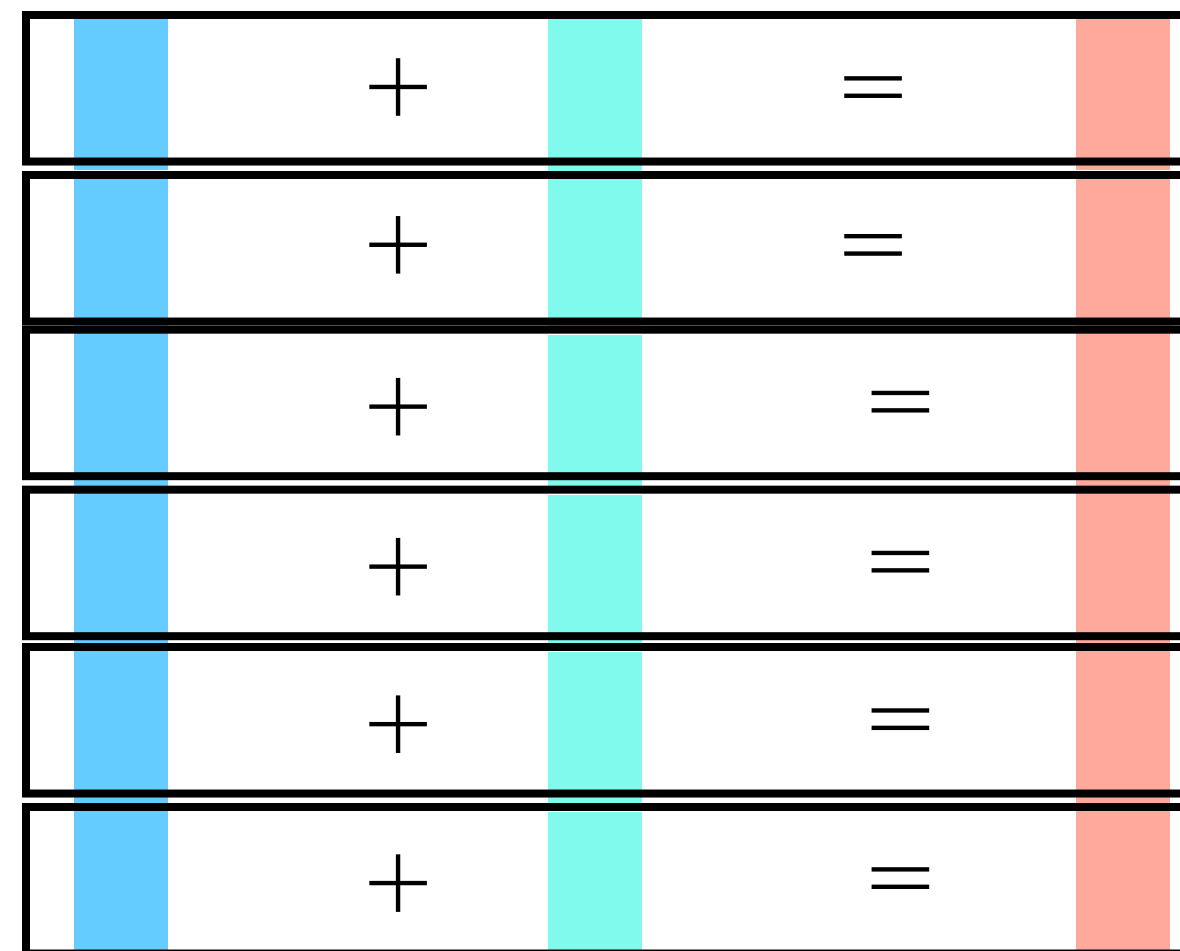


Parallel execution

Kernel

- **Kernel** - a function executed on the GPU by many threads
- Each part computes a portion of the result (same program, different data):

```
__global__ void vec_add(float* a, float* b, float* c, int n) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n) c[i] = a[i] + b[i];  
}  
  
vec_add<<<num_blocks, block_size>>>>(a, b, c, n);
```



A

B

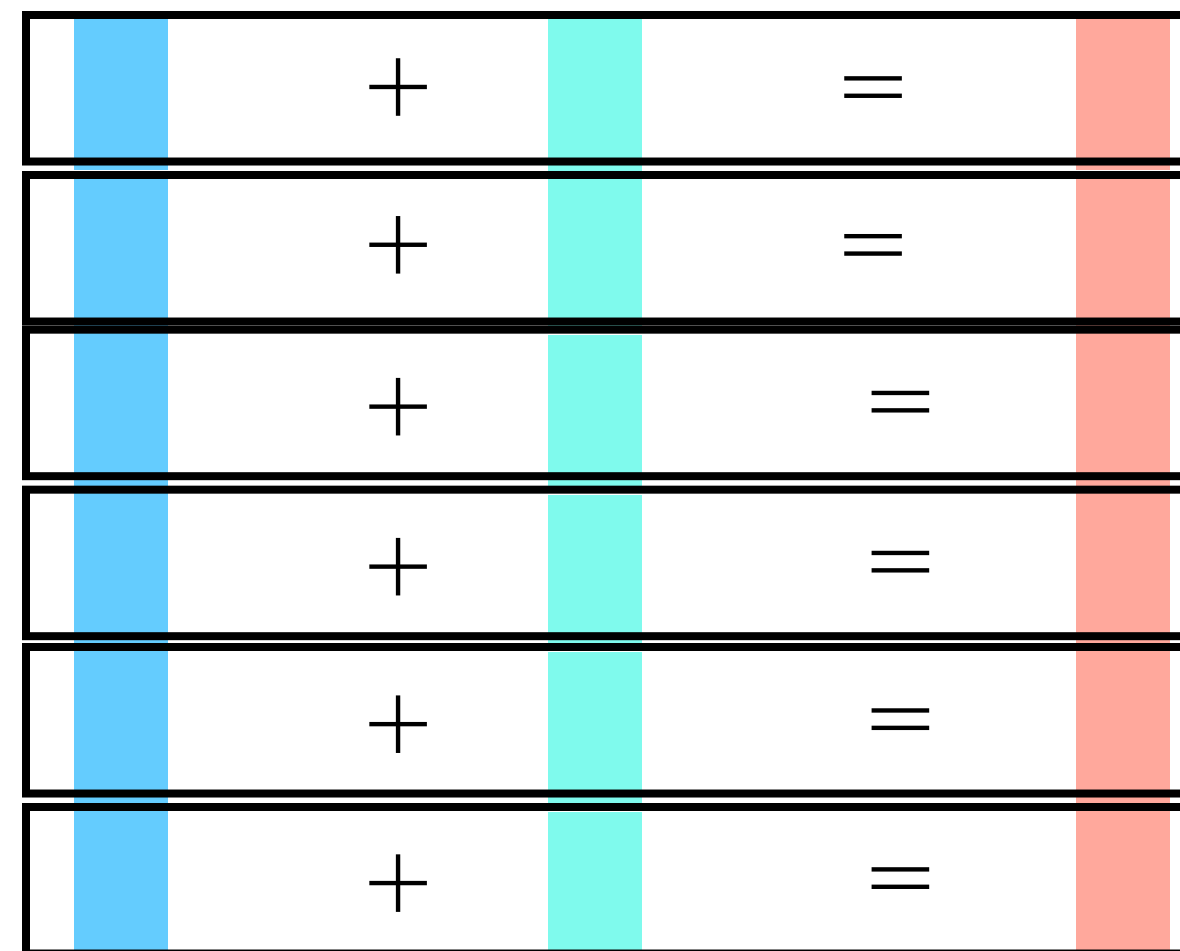
C

Parallel execution

Kernel

- **Kernel** - a function executed on the GPU by many threads
- Each part computes a portion of the result (same program, different data):

```
__global__ void vec_add(float* a, float* b, float* c, int n) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n) c[i] = a[i] + b[i];  
}  
  
vec_add<<<num_blocks, block_size>>>>(a, b, c, n);
```



A

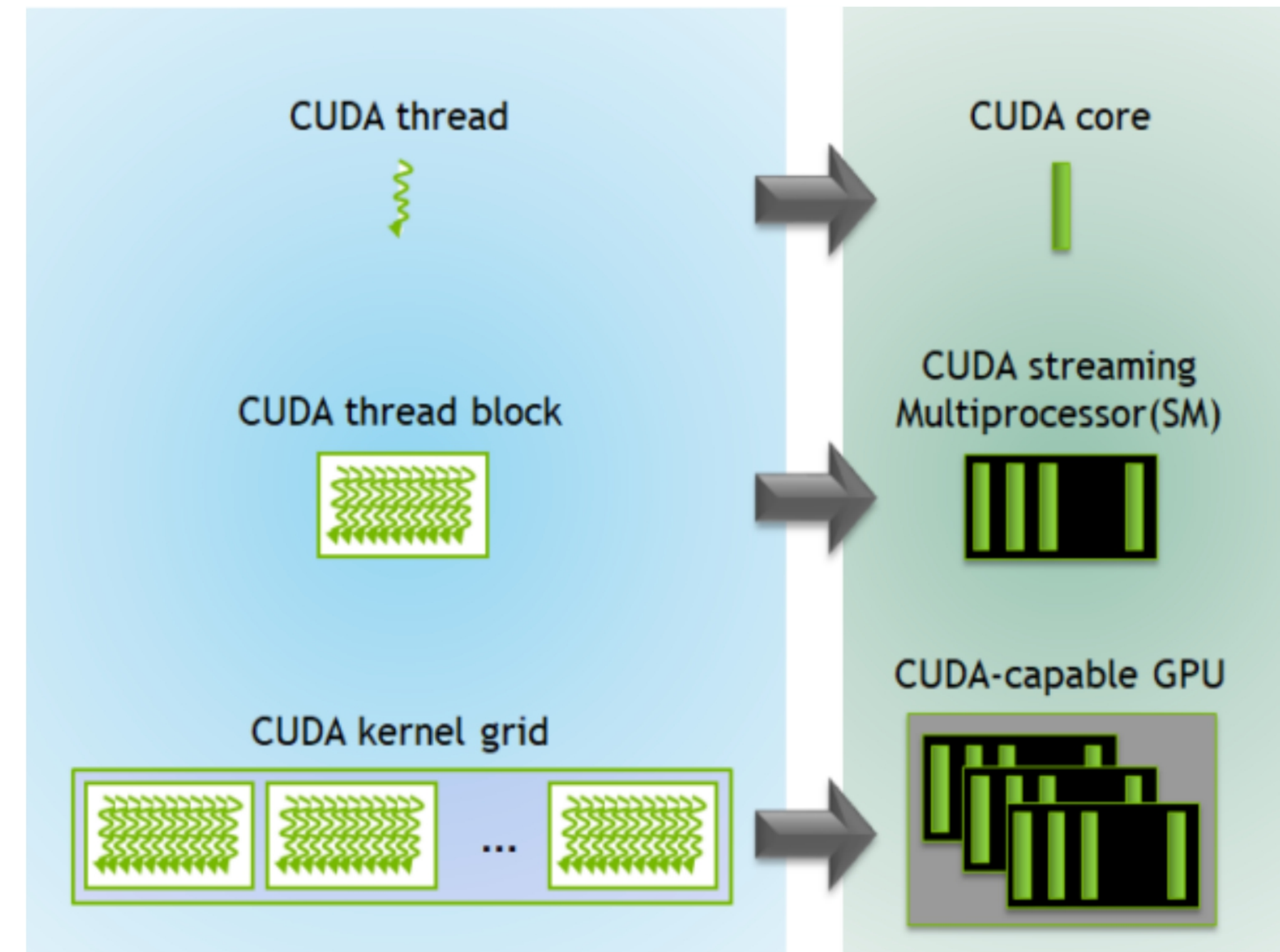
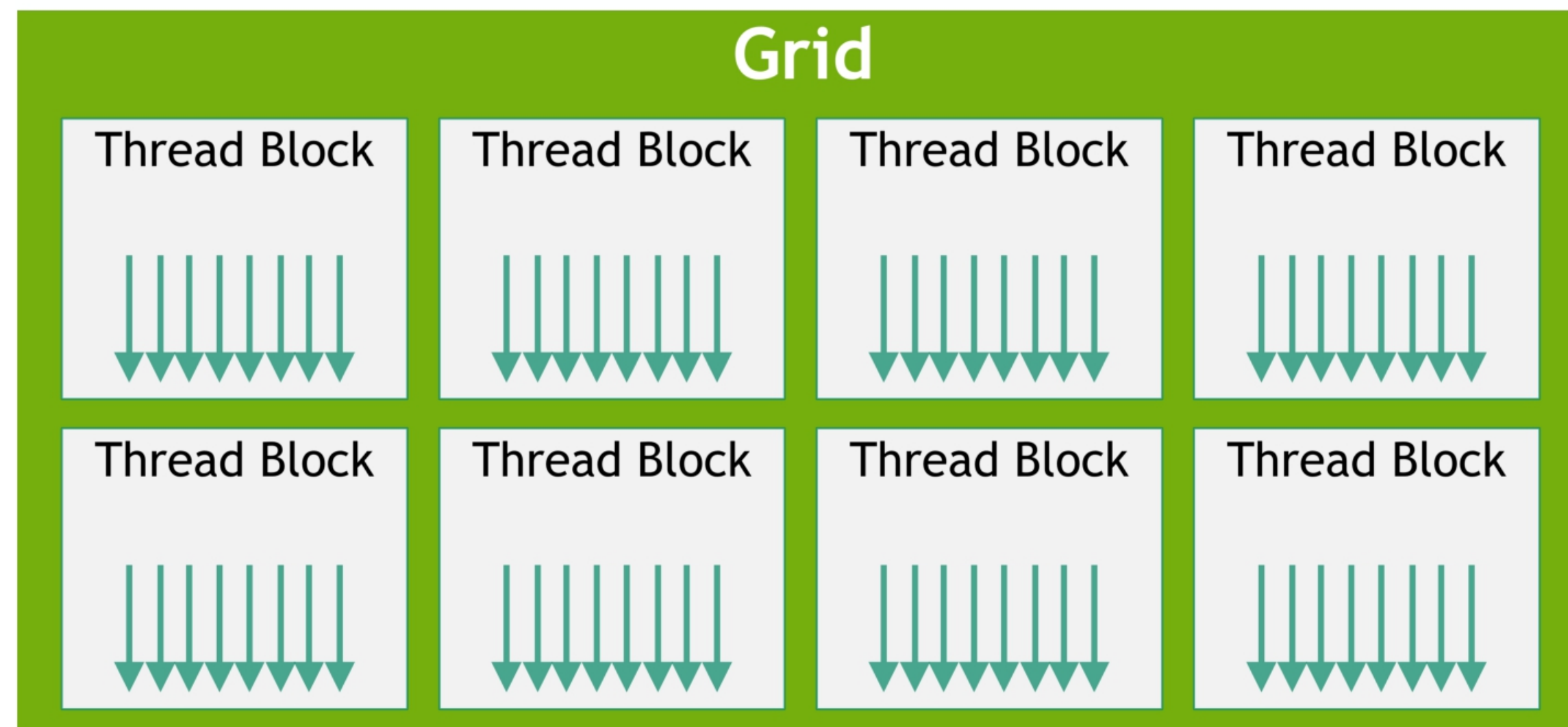
B

C

Parallel execution

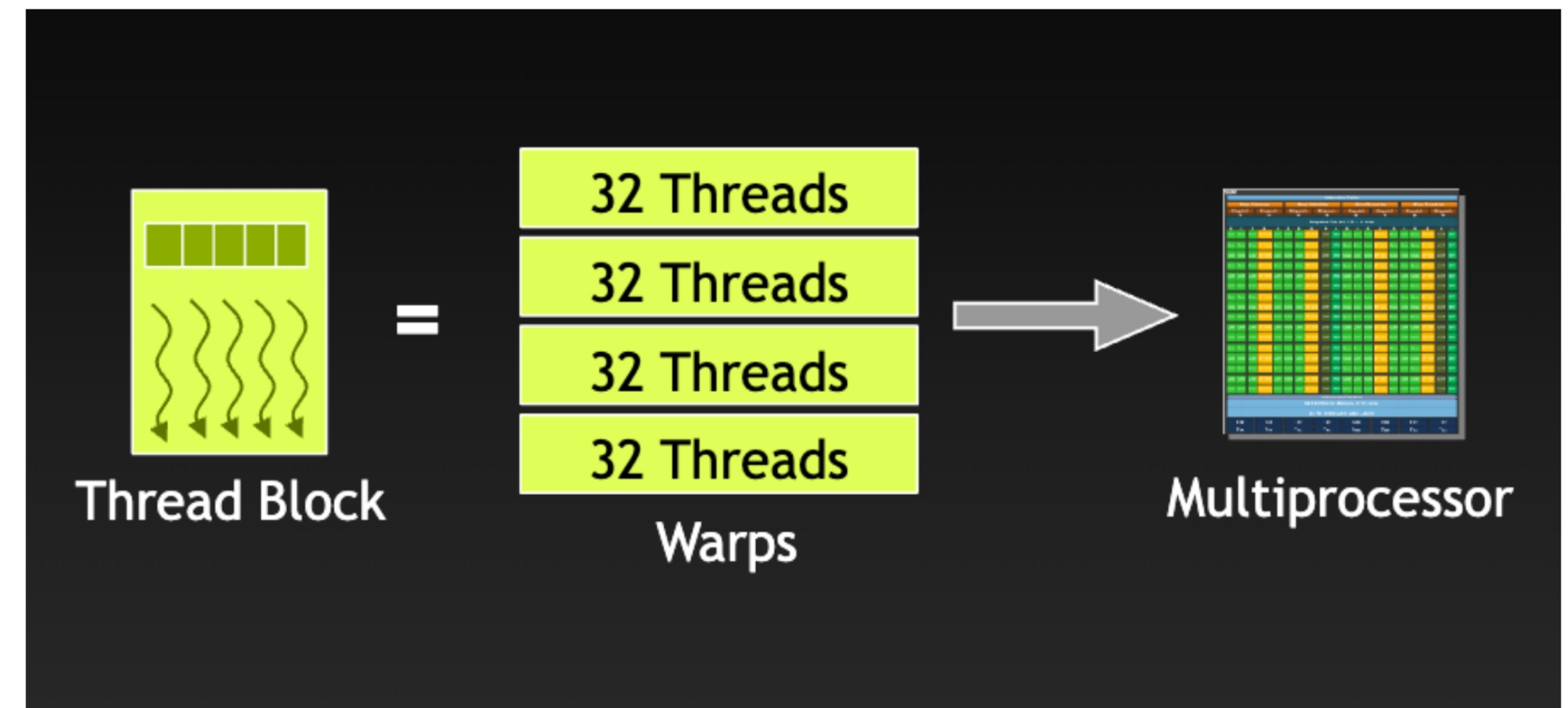
Execution Hierarchy

- Kernel is launched with multiple **threads**
- **Threads** are grouped into **thread blocks**
 - Each thread block is launched on a single SM
 - Number of thread blocks (threads) on SM is limited
 - Thread blocks are required to execute independently of one another
 - Threads within a thread block can cooperate
- Thread blocks form a **grid** for a kernel launch



Physical Level for CUDA model

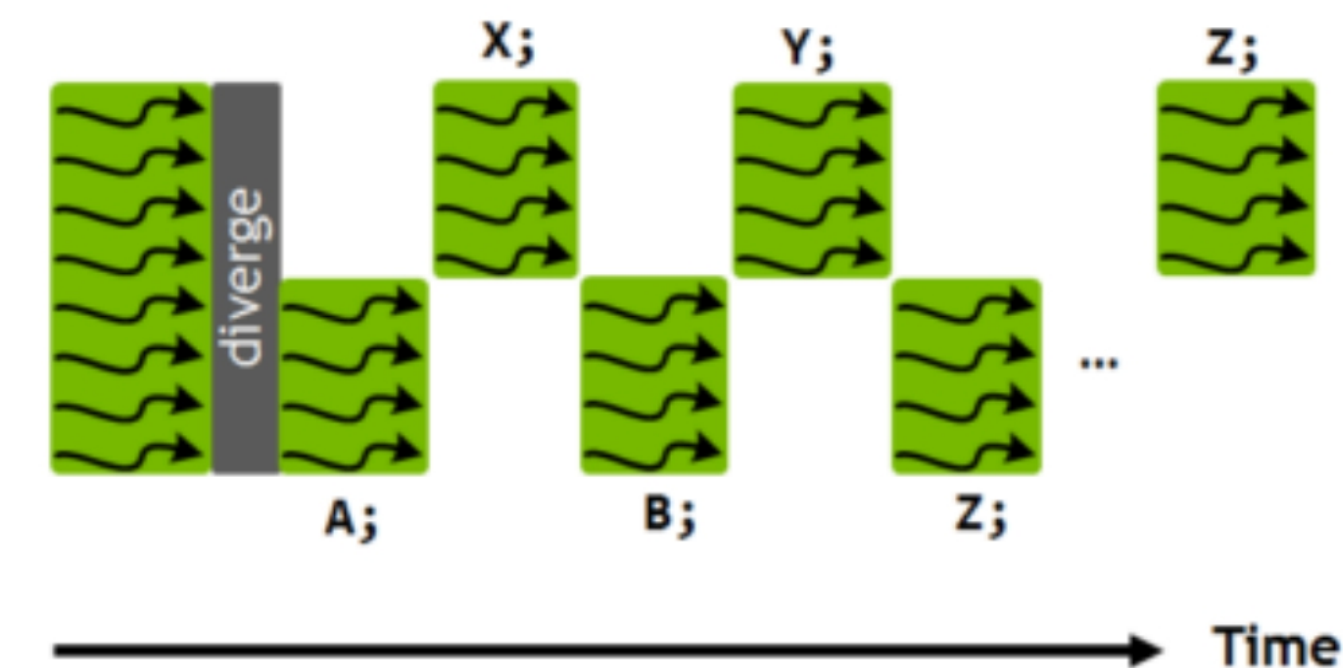
- Threads in a thread block are guaranteed to launch on the same SM
- Threads are organized and launched in a **warp** - a group of 32 threads *sharing the same instruction pointer*
- Threads in a warp can also communicate
- There can be multiple warps in a thread block



Physical Level for CUDA model

- Threads are organized and launched in a **warp** - a group of 32 threads *sharing the same instruction pointer*
- $\implies$ Threads in a warp execute the same instruction every clock cycle
- SIMT (Single Instructions Multiple Threads Paradigm)
- If there is a **branch divergence** within a warp, execution of each branch is serialized (**affects performance**)

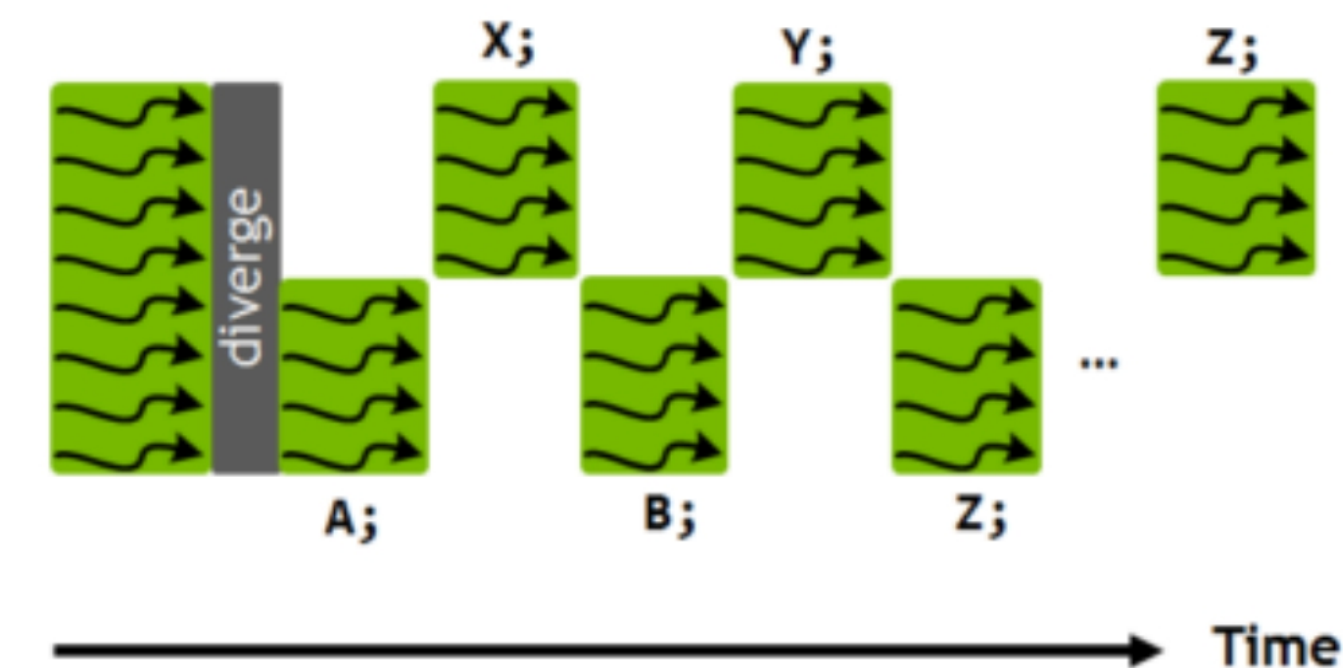
```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```



Physical Level for CUDA model

- Threads are organized and launched in a **warp** - a group of 32 threads *sharing the same instruction pointer*
- $\implies$ Threads in a warp execute the same instruction every clock cycle
- SIMT (Single Instructions Multiple Threads Paradigm)
- If there is a **branch divergence** within a warp, execution of each branch is serialized (**affects performance**)

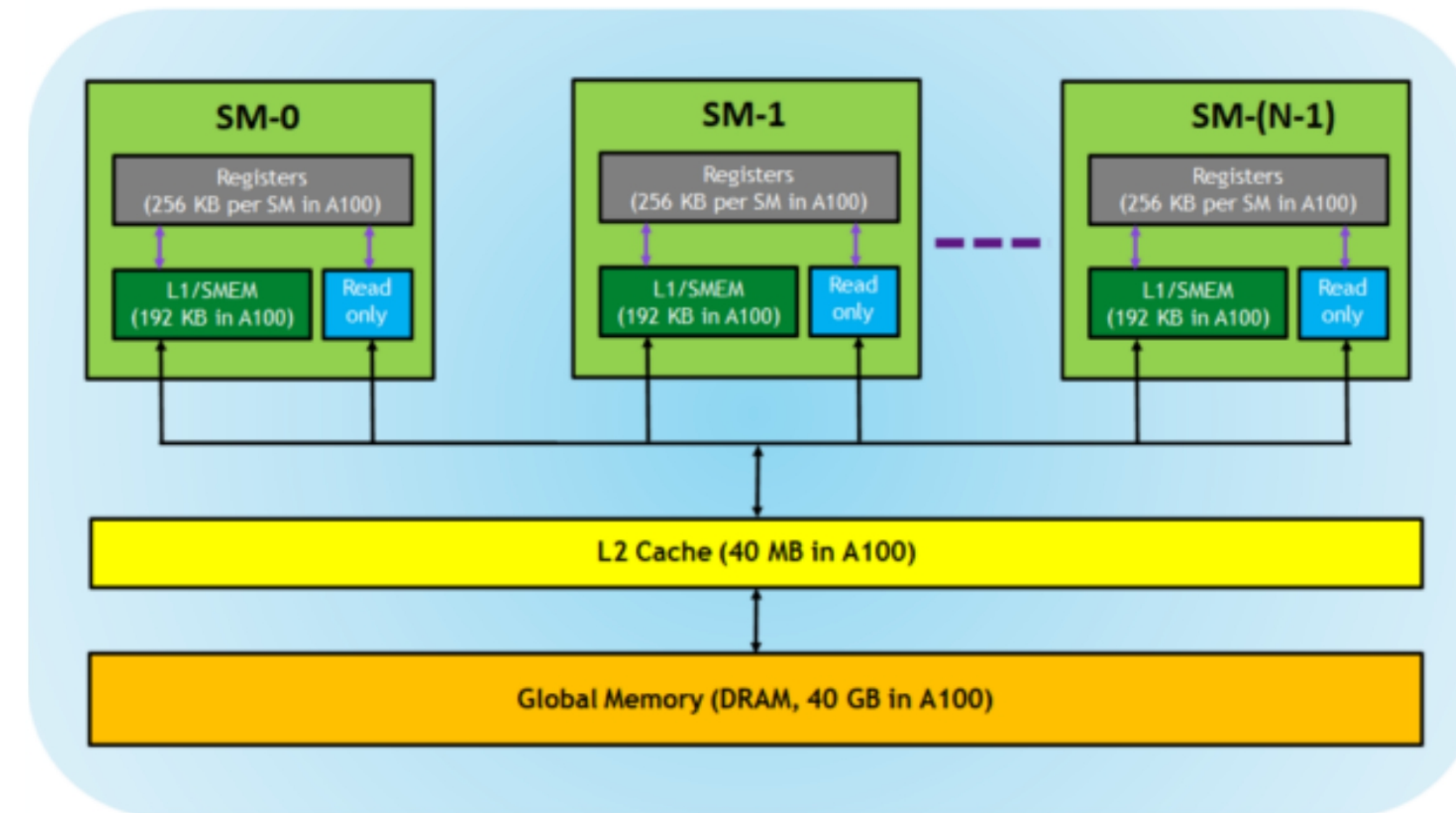
```
if (threadIdx.x < 4) {  
    A;  
    B;  
} else {  
    X;  
    Y;  
}  
Z;
```



Memory Hierarchy

GPU Memory Hierarchy

- GPU memory consists of several layers with varying scope, capacity and cost

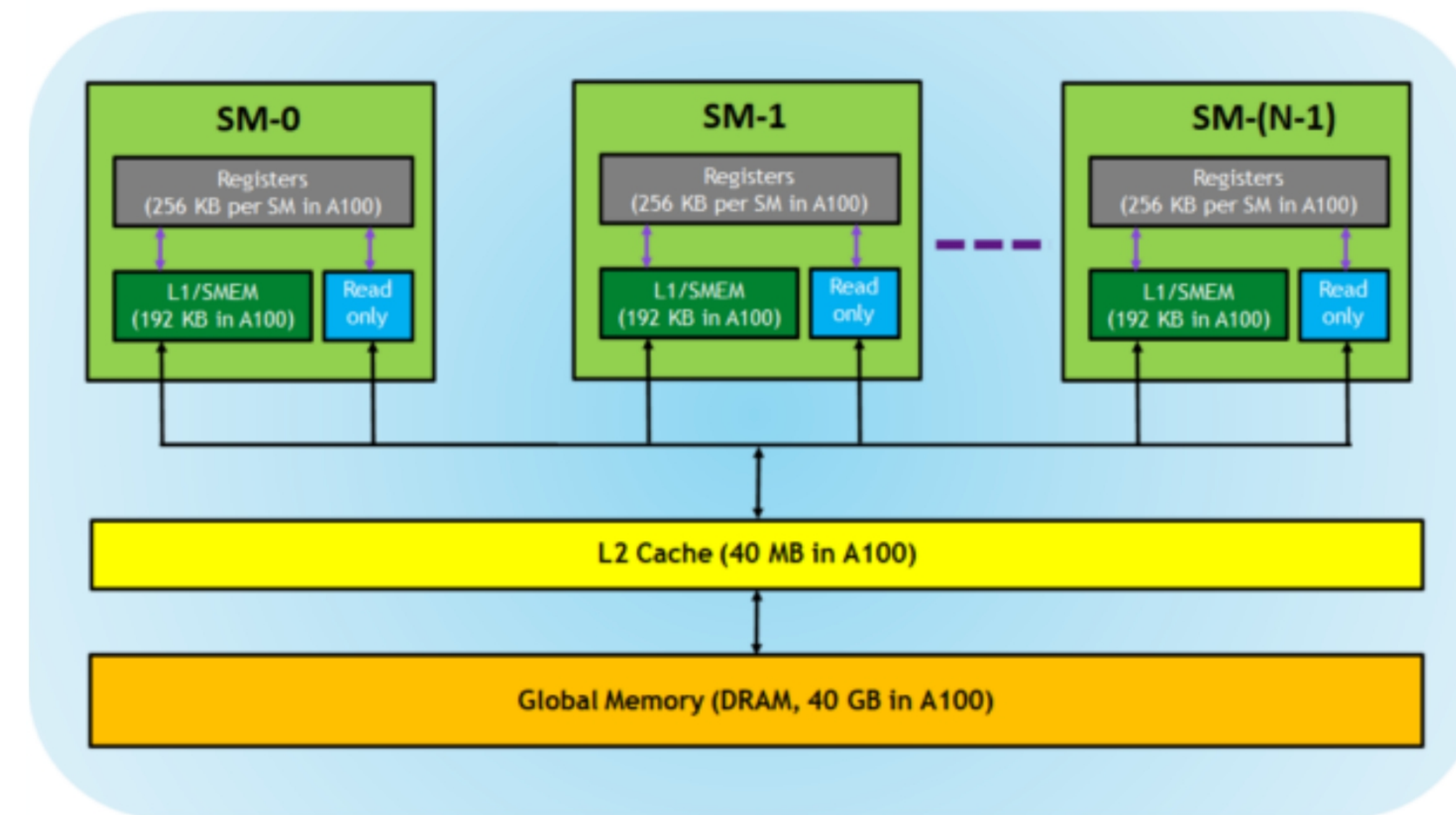


Memory Hierarchy of NVIDIA A100 40GB

GPU Memory Hierarchy

- GPU memory consists of several layers with varying scope, capacity and cost
- **Global** memory (HBM, high-bandwidth memory) - a lot, but **slow**. Frequently accessed elements are stored in L2 cache
- **Shared** memory - shared across SM - *much* smaller, but **faster**
- Runtime variables are stored in **registers** - **thread-specific storage**

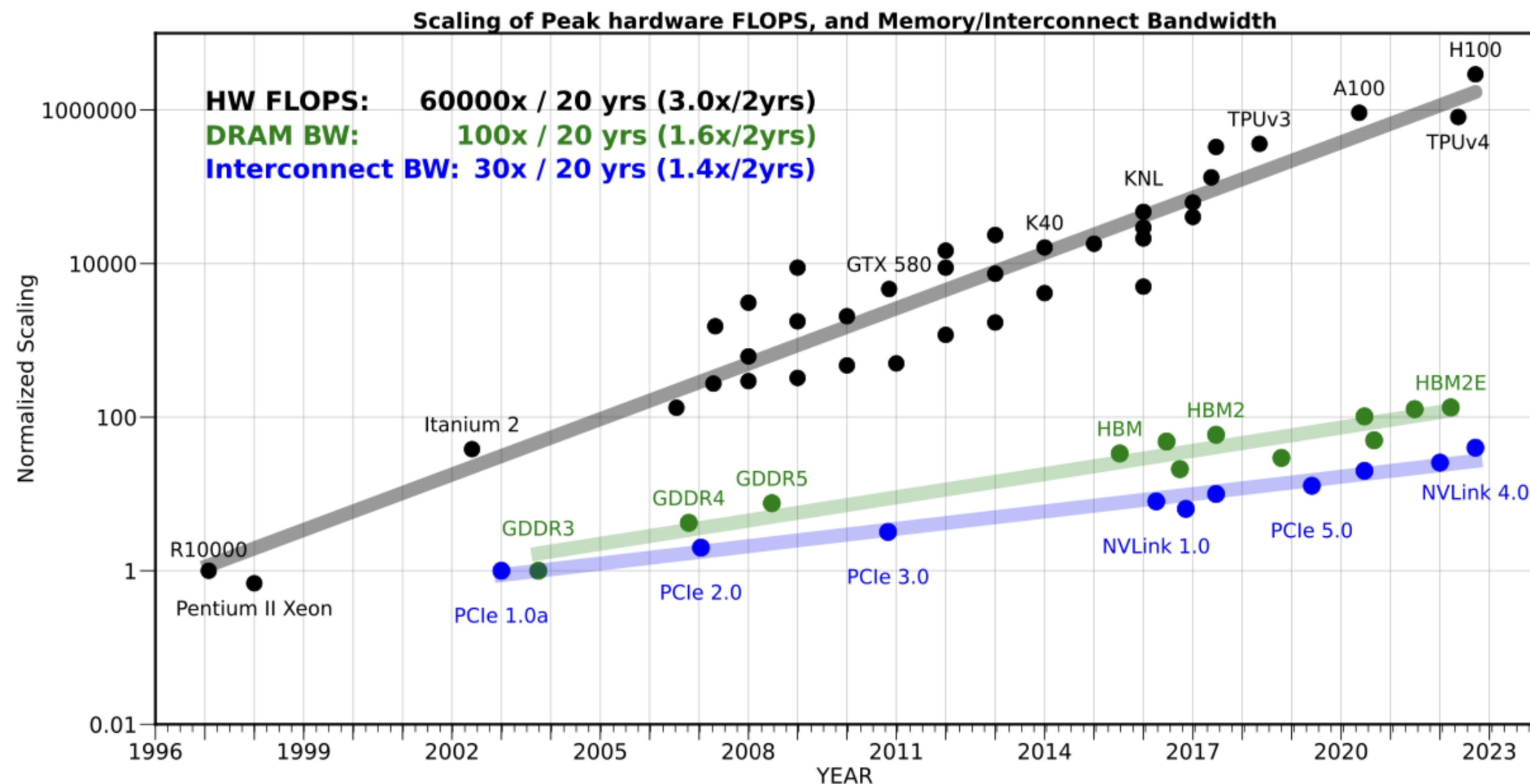
Memory access can bottleneck execution



Memory Hierarchy of NVIDIA A100 40GB

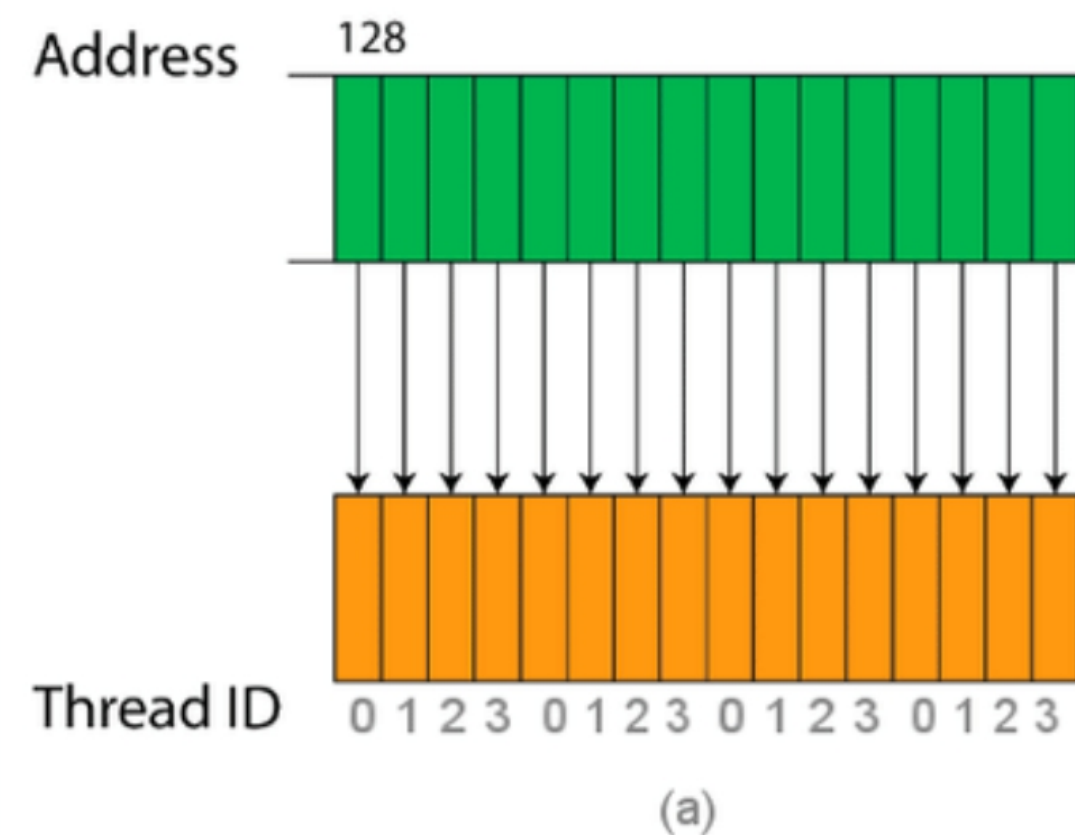
GPU Memory Trends

- The bandwidth has been increased slower than compute over the years
- $\Rightarrow$ usually we are bottlenecked by memory

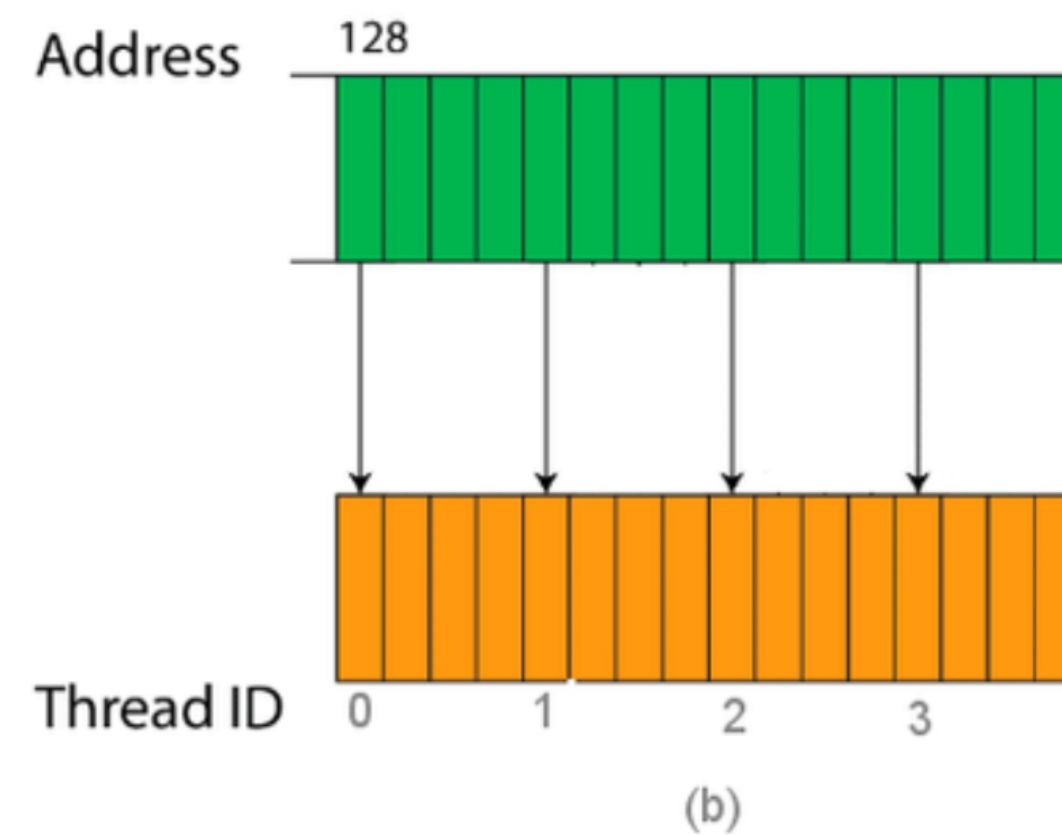


GPU Global Memory (HBM)

- Accessible for all threads, slowest, but largest in size
- Accesses are performed in transactions
- ***Coalesced memory access*** within a warp is grouped into a single transaction



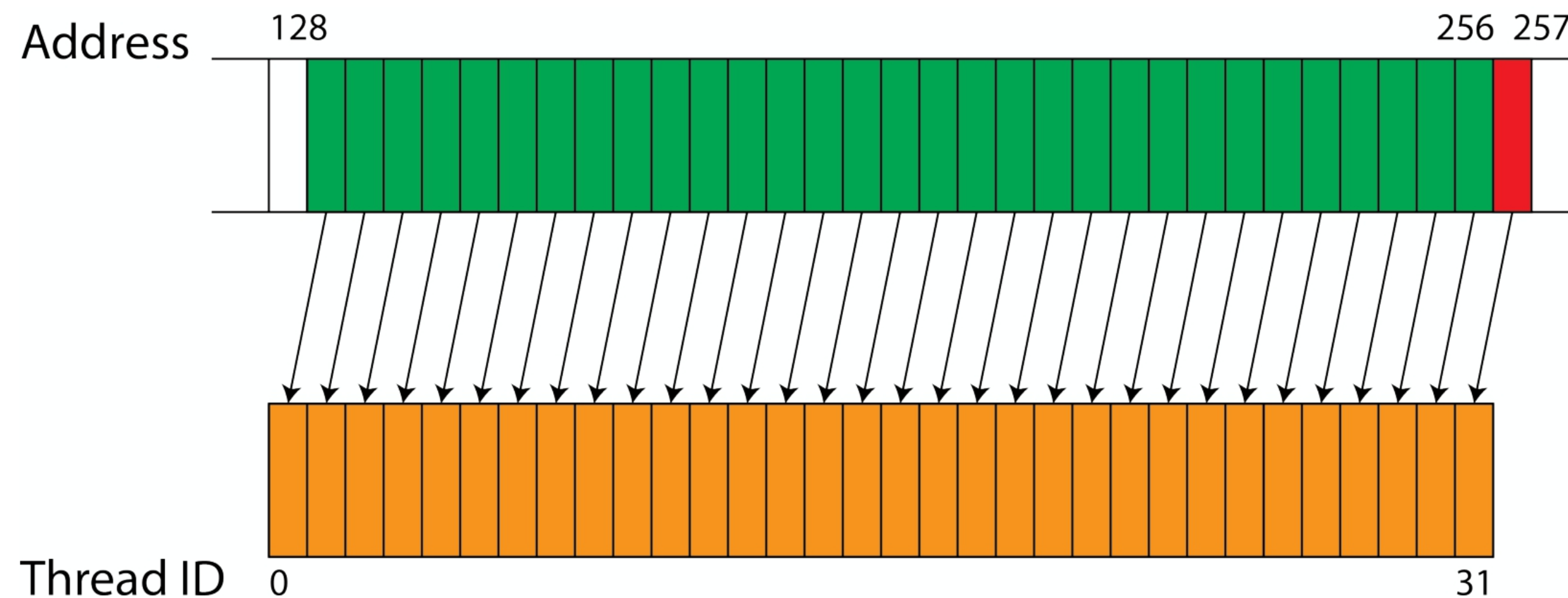
(a) coalesced memory access



(a) uncoalesced memory access. Multiple transactions

GPU Global Memory (HBM)

- Accessible for all threads, slowest, but largest in size
- Accesses are performed in transactions
- ***Coalesced memory access*** within a warp is grouped into a single transaction
- Addresses must be aligned by 128 bytes for coalesced reading



Latency Hiding

- When having multiple warps, we can hide the huge latency of IO operations by switching between them
- When one warp waits, warp scheduler can run another ready warp
- Switching between warp is near free, so warp not waiting for the IO can compute while others can wait



GPU Shared Memory (SMEM)

- Shared Memory resides on SM $\implies$ much lower latency than global memory
- Accessible for all threads within a thread block
 - On newer architectures (from Hopper) we can communicate between thread blocks via Distributed Shared Memory protocol
- Threads can cooperate via SHMEM

GPU Shared Memory (SMEM) usage example

- Threads can cooperate via SHMEM
- In matrix multiplication, each thread block can compute a tile of the output matrix
- Let each thread compute a single element of the output tile
- The inputs elements are reused, so we can load them once to SMEM
- IO is reduced proportionally to tile size

